

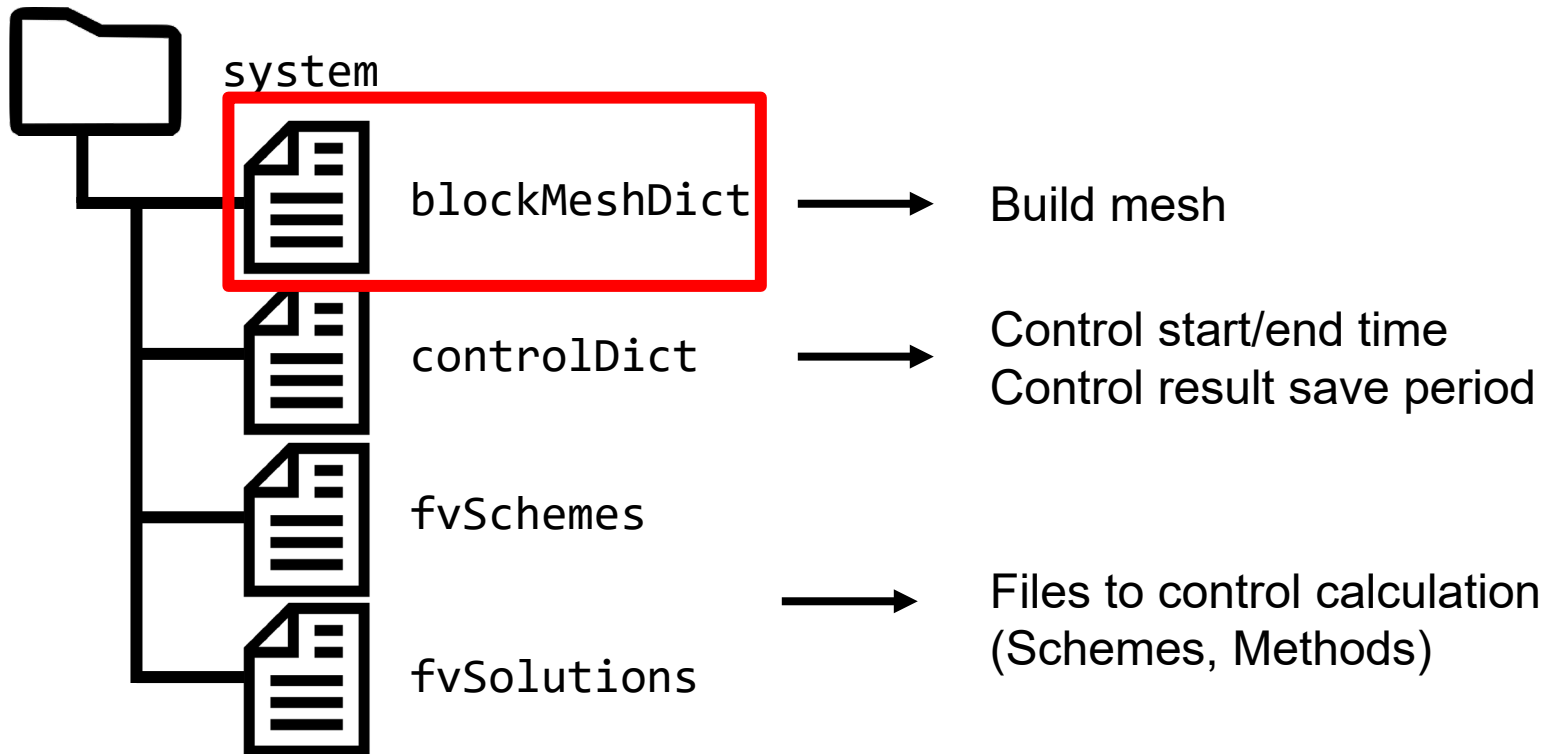
OpenFOAM blockMesh 격자 생성 가이드

2026 한국연소학회 가을 Tutorial
Next-Gen Combustion

OpenFOAM

- **Step 1: Pre-processing**

- File tree



OpenFOAM

● Step 1-1 : Mesh Generation

- The mesh generator supplied with OpenFOAM : blockMesh

```
/*-----*- C++ -*-----*\
//
// \V\  F ield      | OpenFOAM: The Open Source CFD Toolbox
//  \V\  O peration | Website:  https://openfoam.org
//  \V\  A nd       | Version:   12
//  \V\  M anipulation
//
FoamFile
{
    format      ascii;
    class       dictionary;
    object      blockMeshDict;
}
// *****

convertToMeters 0.1;

vertices
(
    (0 0 0)
    (1 0 0)
    (1 1 0)
    (0 1 0)
    (0 0 0.1)
    (1 0 0.1)
    (1 1 0.1)
    (0 1 0.1)
);

blocks
(
    hex (0 1 2 3 4 5 6 7) (20 20 1) simpleGrading (1 1 1)
);

boundary
(
    movingWall
    {
        type wall;
        faces
        (
            (3 7 6 2)
        );
    }
);
```

Command Line

```
cd 1.cavity/system
```

```
vi blockMeshDict
```

● Step 1-1 : Mesh Generation

```
/*----- C++ -----*/
//
//      F i e l d
//      O p e r a t i o n
//      A n d
//      M a n i p u l a t i o n
//
//      OpenFOAM: The Open Source CFD Toolbox
//      Website:  https://openfoam.org
//      Version:  12
//
FoamFile
{
    format      ascii;
    class       dictionary;
    object      blockMeshDict;
}
// *****

convertToMeters 0.1;

vertices
(
    (0 0 0)
    (1 0 0)
    (1 1 0)
    (0 1 0)
    (0 0 0.1)
    (1 0 0.1)
    (1 1 0.1)
    (0 1 0.1)
);

blocks
(
    hex (0 1 2 3 4 5 6 7) (20 20 1) simpleGrading (1 1 1)
);

boundary
(
    movingWall
    {
        type wall;
        faces
        (
            (3 7 6 2)
        )
    }
);
```

General procedure for mesh-generation by using *blockMesh*

1. Define the length scale of domain (*convertToMeters*)
2. Generate *vertices*
3. Connect the *vertices*, and generate the *block* (typically *block* is hexagonal structure such that each block requires 8 vertices)
4. Decide the number of cells of *blocks* for each direction
5. Naming each face of block (hexagonal structure consists of 6 faces), and define the type of each face (e.g., wall, patch, empty, wedge, ...)
6. Go back to the reference directory, and type *blockMesh*
7. Check the mesh by using several visualization tools (e.g., Paraview)

OpenFOAM

● Step 1-1 : Mesh Generation

1) Define the length scale of domain (convertToMeters)

```
/*-----*-- C++ --*-----*/
=====
\ \   F i e l d           | OpenFOAM: The Open Source CFD Toolbox
 \ \   O peration        | Website:  https://openfoam.org
  \ \   A nd             | Version:  12
   \ \   M anipulation    |
/*-----*/

FoamFile
{
    format      ascii;
    class       dictionary;
    object      blockMeshDict;
}

// *****

convertToMeters 0.1;

vertices
(
    (0 0 0)
    (1 0 0)
    (1 1 0)
    (0 1 0)
    (0 0 0.1)
    (1 0 0.1)
    (1 1 0.1)
    (0 1 0.1)
);
```

convertToMeters: Scaling factor for the vertex coordinates.
For instance,
convertToMeters 0.1; (1 0 0) refers to (0.1 m, 0 m, 0 m)
convertToMeters 0.001; (1 1 1) refers to (1 mm, 1mm, 1mm)

OpenFOAM

● Step 1-1 : Mesh Generation

2) Generate vertices

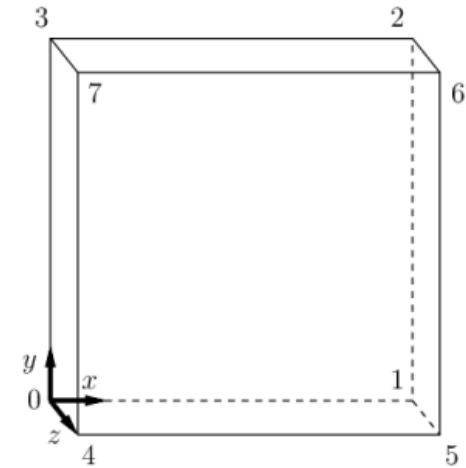
```
/*-----*-- C++ --*-----*/
//====
// \  F i e l d      | OpenFOAM: The Open Source CFD Toolbox
//  \ O p e r a t i o n | Website: https://openfoam.org
//   \ A n d           | Version: 12
//    \ M a n i p u l a t i o n |
//====
FoamFile
{
    format      ascii;
    class       dictionary;
    object      blockMeshDict;
}
// *****
convertToMeters 0.1;

vertices
(
    (0 0 0)
    (1 0 0)
    (1 1 0)
    (0 1 0)
    (0 0 0.1)
    (1 0 0.1)
    (1 1 0.1)
    (0 1 0.1)
);
```

vertices: generate the list of vertex coordinates
Each vertex is automatically numbered **starting from 0**.

For instance,

(0 0 0) // **vertex index 0**
(1 0 0) // **vertex index 1**
(1 1 0) // **vertex index 2**
(0 1 0) // **vertex index 3**
(0 0 0.1) // **vertex index 4**
(1 0 0.1) // **vertex index 5**
(1 1 0.1) // **vertex index 6**
(0 1 0.1) // **vertex index 7**



OpenFOAM

● Step 1-1 : Mesh Generation

3) Connect the vertices, and generate the block

4) Decide the number of cells of blocks

```
blocks
(
  hex (0 1 2 3 4 5 6 7) (20 20 1) simpleGrading (1 1 1)
);
```

The length ratio of first to last cell in each direction. (1 1 1) means uniform grid size

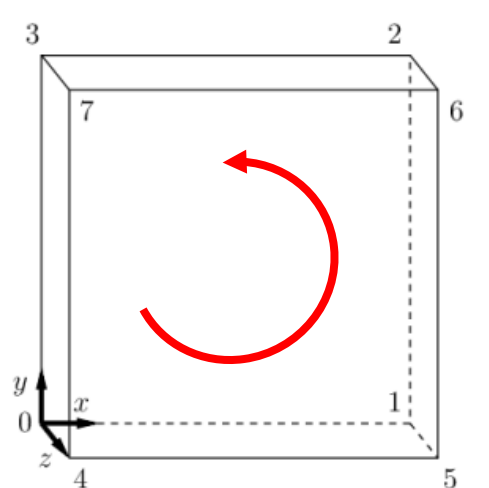
Number of cells in x, y, z directions, respectively. This is a 2-D configuration so that only 1 cell is placed in z-directions

List of eight vertices (index of vertex is given in accordance with previous slide)

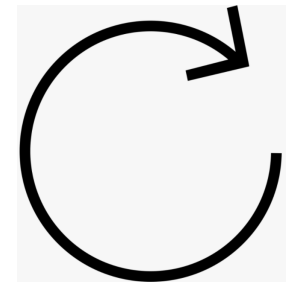
type: hexagonal block

blocks: each block has a local coordinate system that **list of vertices must be right-handed order.**

For instance, arbitrary vertex list is not allowed, such as hex (2 4 3 1 4 5 7 6)



Ex)



hex (4 7 6 5 0 3 2 1)

OpenFOAM

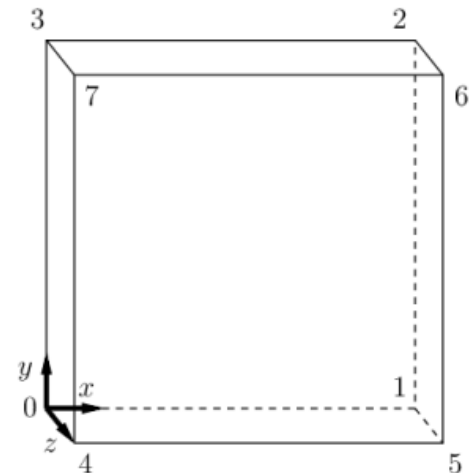
● Step 1-1 : Mesh Generation

5) Naming each face of block and define the type of each face

```
blocks
(
    hex (0 1 2 3 4 5 6 7) (20 20 1) simpleGrading (1 1 1)
);

boundary
(
    movingWall
    {
        type wall;
        faces
        (
            (3 7 6 2)
        )
    }
    fixedWalls
    {
        type wall;
        faces
        (
            (0 4 7 3)
            (2 6 5 1)
            (1 5 4 0)
        )
    }
    frontAndBack
    {
        type empty;
        faces
        (
            (0 3 2 1)
            (4 5 6 7)
        )
    }
);
```

Since we generate one hexagonal block, we need to specify the name & type of 6 faces. Of course, each face consists of 4 vertices, such as (3 7 6 2)



OpenFOAM

● Step 1-1 : Mesh Generation

5) Naming each face of block and define the type of each face

```
blocks
(
  hex (0 1 2 3 4 5 6 7) (20 20 1) simpleGrading (1 1 1)
);

boundary
(
  movingWall
  {
    type wall;
    faces
    (
      (3 7 6 2)
    )
  }
  fixedWalls
  {
    type wall;
    faces
    (
      (0 4 7 3)
      (2 6 5 1)
      (1 5 4 0)
    )
  }
  frontAndBack
  {
    type empty;
    faces
    [
      (0 3 2 1)
      (4 5 6 7)
    ];
  }
);
```

Name : movingWall

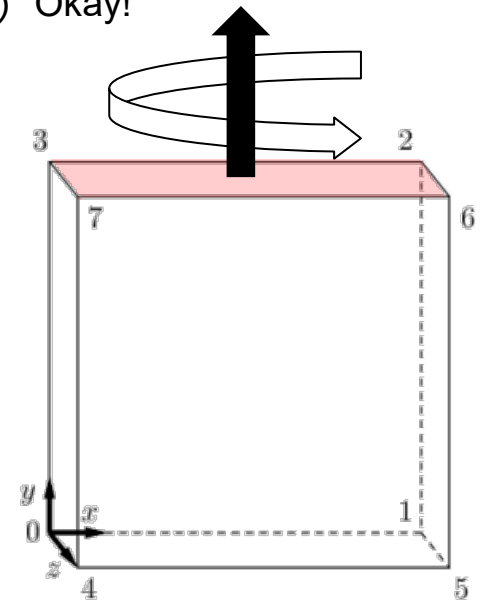
Type: wall

(wall type of boundary does not allow for fluid to pass across the face)

faces: consists of four vertices (3 7 6 2)

(NOTE! : the order in which the vertices are given must be such that, **looking from inside the block and starting with any vertex**, the face must be traversed in **a clockwise direction** to define the other vertices

(7 6 2 3) Okay!



OpenFOAM

● Step 1-1 : Mesh Generation

5) Naming each face of block and define the type of each face

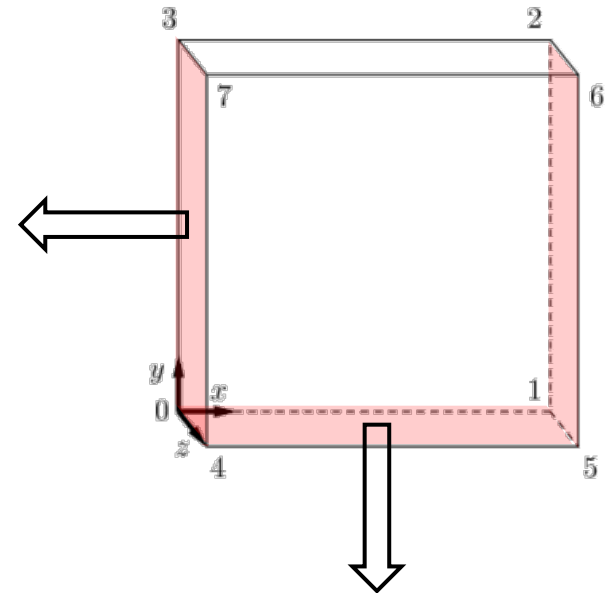
```
blocks
(
    hex (0 1 2 3 4 5 6 7) (20 20 1) simpleGrading (1 1 1)
);

boundary
(
    movingWall
    {
        type wall;
        faces
        (
            (3 7 6 2)
        );
    }
    fixedWalls
    {
        type wall;
        faces
        (
            (0 4 7 3)
            (2 6 5 1)
            (1 5 4 0)
        );
    }
    frontAndBack
    {
        type empty;
        faces
        [
            (0 3 2 1)
            (4 5 6 7)
        ];
    }
);
```

Name : fixedWalls

Type: wall

faces: three faces consist of one boundary.
This is because the properties of these remaining walls are the same (i.e., $u = \text{no slip condition}$, $p = \text{zero Gradient condition}$)



OpenFOAM

● Step 1-1 : Mesh Generation

5) Naming each face of block and define the type of each face

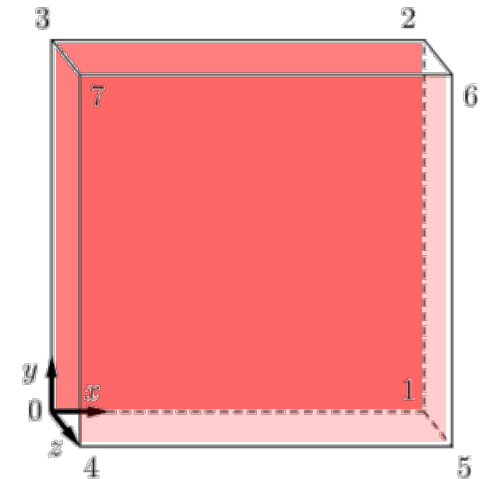
```
blocks
(
    hex (0 1 2 3 4 5 6 7) (20 20 1) simpleGrading (1 1 1)
);

boundary
(
    movingWall
    {
        type wall;
        faces
        (
            (3 7 6 2)
        );
    }
    fixedWalls
    {
        type wall;
        faces
        (
            (0 4 7 3)
            (2 6 5 1)
            (1 5 4 0)
        );
    }
    frontAndBack
    {
        type empty;
        faces
        (
            (0 3 2 1)
            (4 5 6 7)
        );
    }
);
```

Name : fixedWalls

Type: wall

faces: Two faces consist of one boundary. By specifying **these faces** to “empty” condition, together with **placing only 1 cell in z-direction**, OpenFOAM does not solve the discretization equation along the z-direction (2-D problem)



● Step 1-1: Mesh generation

Command Line

```
:wq  
cd ../  
blockMesh
```

General procedure for mesh-generation by using *blockMesh*

6. Go back to the reference directory, and type *blockMesh*

```
Writing polyMesh  
-----  
Mesh Information  
-----  
boundingBox: (0 0 0) (0.1 0.1 0.01)  
nPoints: 882  
nCells: 400  
nFaces: 1640  
nInternalFaces: 760  
-----  
Patches  
-----  
patch 0 (start: 760 size: 20) name: movingWall  
patch 1 (start: 780 size: 60) name: fixedWalls  
patch 2 (start: 840 size: 800) name: frontAndBack  
End
```

of cells in x, y = 20, 20

blockMesh: The mesh is generated by running blockMesh command. Any mistakes in the blockMeshDict file are picked up by blockMesh and the resulting error message directs the user to the line in the file where the problem occurred. **There should be no error messages at this stage.**

NOTE! to run blockMesh, you need to be at the reference directory where 0, constant, and system directories are located.
(For instance, 1.cavity directory is the reference directory for this case)